

HyMedia: Cloud-Native Multimedia Sharing Platform

A scalable multimedia platform supporting images, videos, audio and GIF media with advanced search, privacy controls and creator monetization.

Name: Abdulrehman Mohamedrafiq Vohra

Student ID: B00968573

Course: COM682_Cloud_Native_Development

System Overview

HyMedia is a cloud-native multimedia sharing platform designed to run on **Microsoft Azure**. The system enables users to upload, manage, and interact with media such as images, videos, and audio while supporting features like **tagging, privacy controls, sensitive content filtering, and premium creator subscriptions**.

The platform uses a **serverless architecture with REST APIs, scalable cloud storage, and asynchronous processing** to ensure efficient media handling and the ability to scale as user activity and content volume increase.

Problem definition

A multimedia social platform has two very different workload types:

1. Large binary objects (media files)

Images/videos/audio are heavy and require scalable object storage, high throughput, and global delivery.

2. Hot metadata (likes, comments, tags, feed ranking)

Metadata changes constantly and must support fast reads for infinite scrolling feeds and search filtering.

Traditional solutions (single VM/monolith) break under:

- **Burst traffic** (viral content spikes)
- **CPU-heavy processing** (thumbnails, transcoding, waveforms, AI tagging)
- **Global latency** (users far from origin)
- **Complex privacy** (private-to-selected users, premium-only, unlisted links)
- **Safety & compliance** (sensitive content filters + 18+ gating)

This creates a requirement for a **cloud-native architecture** that supports horizontal scaling, asynchronous processing, and managed storage/services.



Cloud-native approach & design principles

Architecture style: Serverless + event-driven “Web–Queue–Worker” pattern

Reason: keep the user-facing path fast and offload slow tasks to a scalable worker.

Key design principles (REST + cloud):

- **Stateless API layer** (scales easily; no server-side session state)
- **Resource-oriented REST design** using HTTP verbs (GET/POST/PATCH/DELETE)
- **Direct-to-Blob uploads using time-limited SAS** (avoid routing huge files through API)
- **Async processing pipeline** using queue + Functions
- **Polyglot persistence:**
 - SQL for identity, ACLs, entitlements, payments/audit
 - NoSQL for flexible feed metadata and evolving content signals
- **Defense-in-depth:** auth, access checks, private URL signing, encryption
- **Service evolution:** API versioning



System scope (what HyMedia supports)

Supported media types

- Images: JPG/JPEG, PNG, GIF, WEBP, HEIC/HEIF, BMP/TIFF (stored as blobs)
- Video: MP4, MOV, WEBM, MKV (stored as blobs; transcoding planned)
- Audio: MP3, WAV, AAC/M4A, OGG (stored as blobs; waveform/transcription planned)

Core user features

- Signup/Login + session cookies
- Upload with caption, hashtags, **tag users**, **tag location**
- Likes + comments
- Infinite scroll home feed + explore page
- Advanced search (filters + tags + location)
- Sensitive content preferences + 18+ gating
- Privacy per post: public/private-to-selected/unlisted/premium-only
- Creator subscriptions & monetization (planned)

Azure high-level architecture

Frontend

- Azure Static Web Apps (SPA UI + theme system)

API gateway

- Azure API Management (auth enforcement, throttling, version routing)

REST API orchestration (CW requirement)

- Azure Logic Apps (HTTP-trigger) as CRUD endpoints and integration hub

Storage

- Azure Blob Storage for media binaries (unstructured data)
- Azure CDN for fast global delivery (public assets; private via signed URLs)

Databases

- Azure SQL: users, sessions, privacy ACL, creator subscriptions, payouts ledger
- Azure Cosmos DB: denormalized feed docs, tags, engagement counts, AI labels

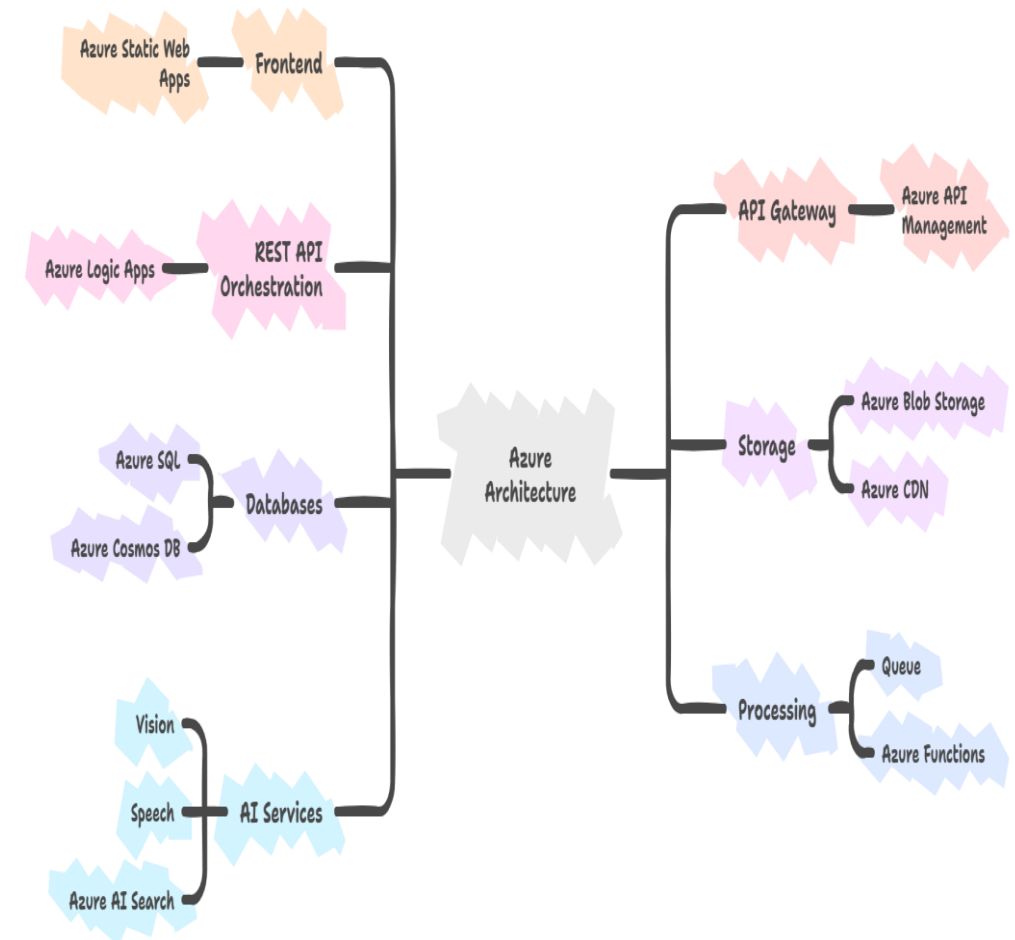
Processing

- Queue (Service Bus or Storage Queue) for async tasks
- Azure Functions for processing (thumbnails/transcode/waveform/AI enrichment)

AI services (planned/advanced)

- Vision for auto-tagging + image embeddings
- Speech for transcription (audio/video)
- Azure AI Search for text + vector search

Azure Architecture for Web Application





Detailed workflow: Upload → process → publish

Fast path (synchronous)

1. Browser sends `POST /api/v1/assets` with metadata (privacy, tags, sensitive flags).
2. Logic App validates identity, creates DB records (SQL/Cosmos), returns:
 1. `assetId`
 2. time-limited **SAS upload URL** for Blob
3. Browser uploads file directly to Blob using SAS (no API bottleneck).

Slow path (asynchronous)

4. Blob upload event (or API) enqueues a message: `{assetId, blobPath, mediaType}`.
5. Functions worker consumes queue:
 - Validate MIME + file signature
 - Generate thumbnail / preview
 - Video transcode variants (planned)
 - Audio waveform extraction (planned)
 - AI tagging + moderation labels (planned)
6. Update Cosmos metadata: `processingStatus=READY`, store derived assets.
7. Feed/search can now surface the content based on privacy + preferences.

This approach scales because upload requests stay lightweight while processing scales independently.



Authentication + session cookies

Auth flow

- `POST /api/v1/auth/signup`
- `POST /api/v1/auth/login`

On login success:

- API issues a **secure session cookie**:
 - `HttpOnly` (not accessible via JS)
 - `Secure` (HTTPS only)
 - `SameSite=Lax/Strict` (CSRF mitigation)

Session data stored in SQL:

- `Sessions(sessionId, userId, createdAt, expiresAt, revokedAt, userAgentHash, ipHash)`

Requests include cookie automatically.

API gateway + Logic Apps validate session on each request (stateless API servers).

Logout

- `POST /api/v1/auth/logout` revokes session record.



Privacy model + enforcement (core differentiator)

Each asset has `visibility`:

- `PUBLIC`
- `PRIVATE_SELECTED`
- `UNLISTED_LINK`
- `CREATOR_PREMIUM`

Private-to-selected implementation

- SQL table `AssetACL(assetId, viewerUserId, permission)`.

Request to view private content:

- API checks ACL:
 - owner always allowed
 - viewer must exist in ACL

Unlisted

- Not searchable/feed
- Accessible via signed share token:
 - `POST /api/v1/assets/{id}/share-links` creates token with expiry

Creator premium

- Viewer must have active subscription entitlement.

This privacy enforcement is applied both:

- when listing feeds/search results
- when returning actual media URLs

Sensitive content | 18+ gating

Each asset stores:

- `isSensitive` (blur/hide controls)
- `isAdult18Plus`

Each user stores preferences:

- `sensitiveMode`: `HIDE` | `BLUR` | `SHOW`
- `adultContentEnabled`: `boolean` (only if verified)

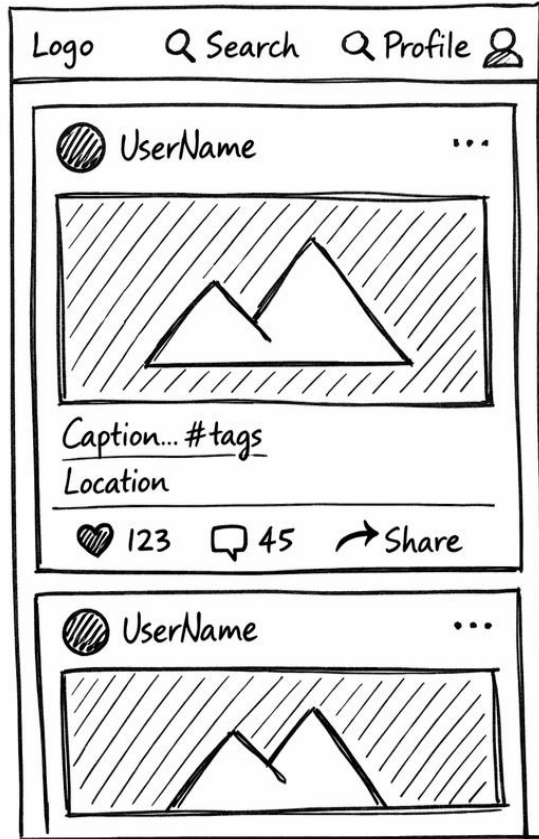
Age verification approach

- MVP: DOB + self-declaration + explicit consent
- Planned: third-party verification / manual review workflow

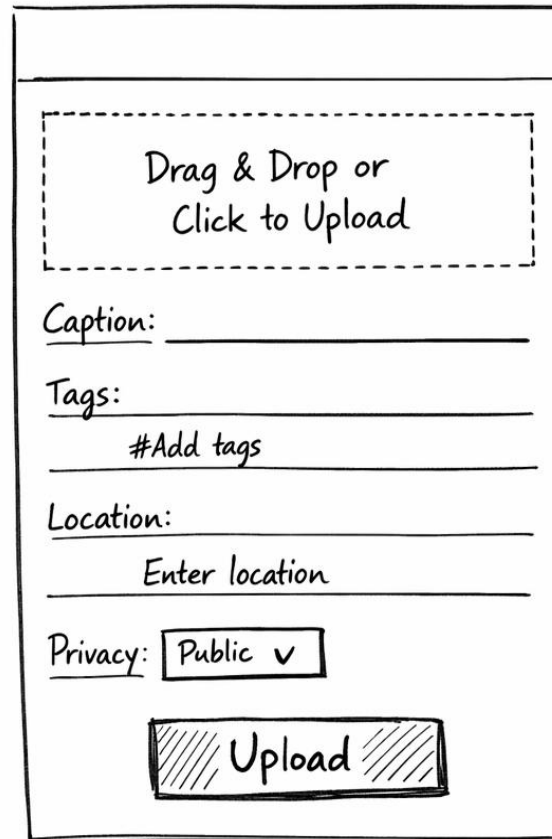
API rules:

- If asset is 18+ and user not verified → block or show warning screen.
- If asset is sensitive → apply user preference (hide/blur/show).

Feed Screen



Upload Screen



- **Main feed**
- Infinite scroll cards:
 - media preview
 - caption + tags
 - like/comment/share
 - badges: sensitive / 18+ / premium
- Upload page fields:
 - file upload + validation message
 - caption
 - hashtags
 - tag users (@mentions)
 - tag location (autocomplete)
 - privacy selector:
 - public / private-to-selected / unlisted / creator-premium
 - sensitive toggle
 - 18+ toggle
 - allow comments on/off
 - upload button
- Two-step upload note:
 - “Create record → SAS URL → direct blob upload → async processing”

Wireframe content

Data model (SQL + Cosmos)

CosmosDB NoSQL Model

SQL tables (ERD)

- Users(UserId, Email, PasswordHash/ExternalId, DOB, Is18Verified, CreatedAt)
- Sessions(SessionId, UserId, ExpiresAt, RevokedAt)
- AssetsCore(AssetId, OwnerId, Visibility, IsSensitive, IsAdult18Plus, CreatedAt)
- AssetACL(AssetId, ViewerUserId, Permission)
- Subscriptions(SubscriptionId, SubscriberId, CreatorId, Status, StartAt, RenewAt)
- Payments(PaymentId, UserId, Amount, Reason, Timestamp)
- PayoutLedger(PayoutId, CreatorId, Amount, RuleId, Timestamp, Status)

Cosmos documents

- AssetFeedDoc:
 - assetId, ownerId, createdAt
 - mediaType, mimeType, blobPublicUrl OR signedUrlRef
 - caption, hashtags, taggedUsers, location
 - likeCount, commentCount
 - aiTags, moderationLabels, embeddingRef
 - processingStatus

Why this split:

- SQL = correctness/permissions/entitlements
- Cosmos = fast feed reads + flexible metadata + denormalized counts

```
{
  "assetId": "a_123",
  "ownerId": "u_001",
  "createdAt": "2026-03-06T10:20:00Z",
  "mediaType": "video",
  "mimeType": "video/mp4",
  "blobPath": "assets/u_001/a_123/original.mp4",
  "derived": {
    "thumbnailPath": "assets/u_001/a_123/thumb.jpg",
    "previewPath": "assets/u_001/a_123/preview.mp4"
  },
  "caption": "Weekend vlog",
  "hashtags": ["#london", "#food"],
  "taggedUsers": ["u_021", "u_099"],
  "location": { "name": "Harrow", "lat": 51.58, "lon": -0.33 },
  "visibility": "PRIVATE_SELECTED",
  "isSensitive": false,
  "isAdult18Plus": false,
  "engagement": { "likeCount": 125, "commentCount": 18 },
  "ai": {
    "tags": ["street", "restaurant"],
    "moderationLabels": ["safe"],
    "embeddingRef": "vec_a_123"
  },
  "processingStatus": "READY"
}
```

REST API design

Auth

- POST /api/v1/auth/signup
- POST /api/v1/auth/login
- POST /api/v1/auth/logout
- GET /api/v1/auth/me

Users

- GET /api/v1/users/{userId}
- PATCH /api/v1/users/me/preferences
Body: theme, sensitiveMode, adultContentEnabled
- POST /api/v1/users/me/verify-age (MVP)

Assets (media posts)

- POST /api/v1/assets
Creates record + returns SAS URL
- PUT /api/v1/assets/{assetId}/blob
(optional: confirm upload complete; triggers enqueue)
- GET /api/v1/assets
Feed list with cursor pagination
Query: cursor, limit, mediaType, tags, location, creatorId
- GET /api/v1/assets/{assetId}
- PATCH /api/v1/assets/{assetId}
caption, hashtags, taggedUsers, location, flags, visibility
- DELETE /api/v1/assets/{assetId}

Privacy & access

- POST /api/v1/assets/{assetId}/acl
Add selected viewers (private-to-selected)
- DELETE /api/v1/assets/{assetId}/acl/{viewerUserId}
- POST /api/v1/assets/{assetId}/share-links
Create unlisted link token
- DELETE /api/v1/assets/{assetId}/share-links/{tokenId}

Likes & comments

- POST /api/v1/assets/{assetId}/likes
- DELETE /api/v1/assets/{assetId}/likes
- POST /api/v1/assets/{assetId}/comments
- GET /api/v1/assets/{assetId}/comments?cursor=&limit=
- DELETE /api/v1/assets/{assetId}/comments/{commentId}
(optional)

Tagging

- POST /api/v1/assets/{assetId}/tags/users (batch tagging)
- POST /api/v1/assets/{assetId}/tags/location

Search

- GET /api/v1/search
Query: q, tags, mediaType, location, from, to, cursor
- POST /api/v1/search/image (planned advanced)
Upload query image → returns similar assets
- POST /api/v1/search/audio (future)
Upload clip → returns best match

UML sequence diagrams

Your module says CREATE + GET is enough to prove CRUD understanding.

UML: CREATE asset (upload)

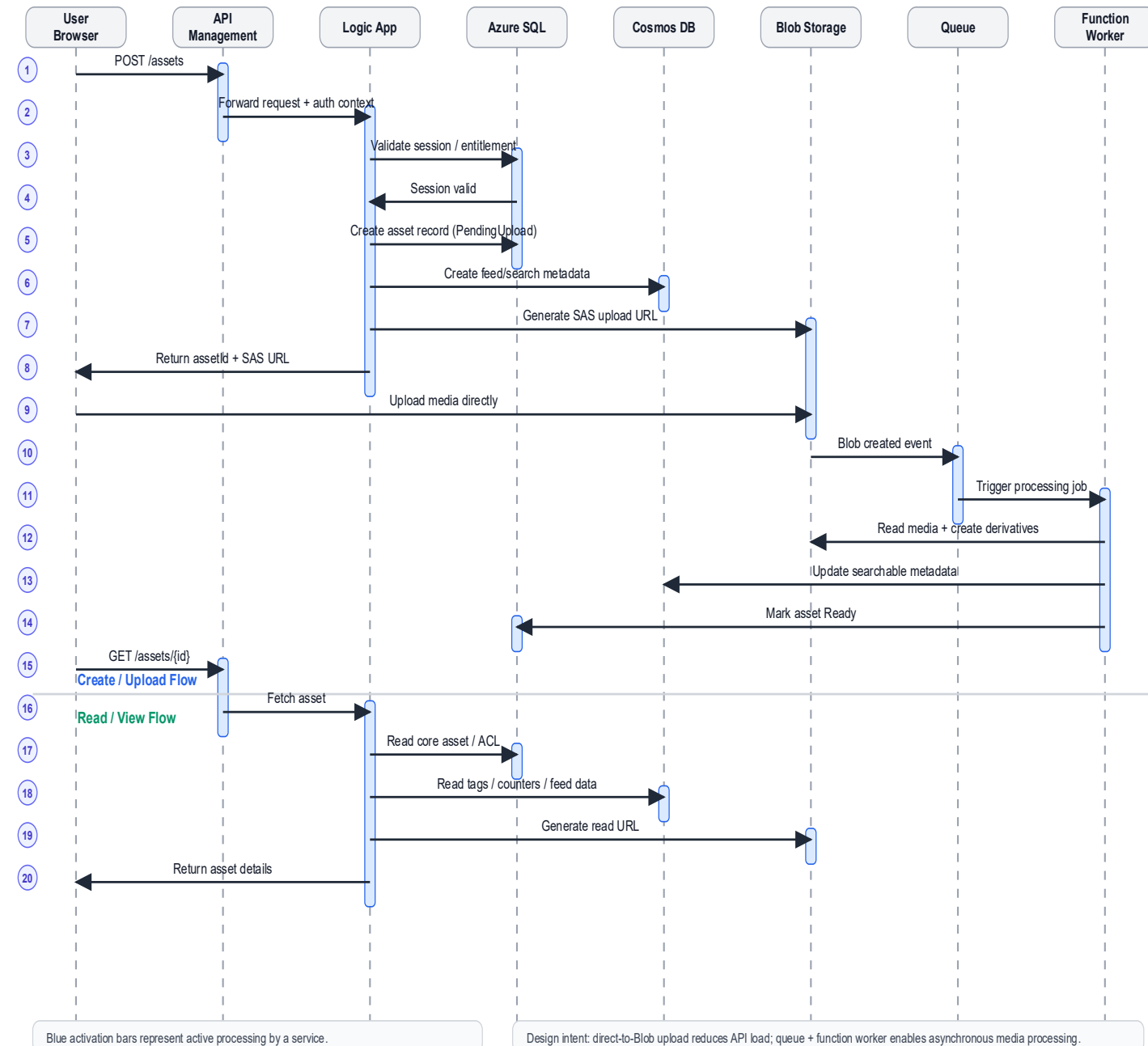
1. User selects file + metadata in browser
2. Browser → APIM: POST /api/v1/assets
3. APIM → LogicApp(CreateAsset): forward request
4. LogicApp validates session cookie, writes:
 - SQL: AssetsCore (+ ACL if private-to-selected)
 - Cosmos: placeholder feed doc
5. LogicApp generates SAS URL for blob upload
6. Response returns assetId + sasUploadUrl
7. Browser uploads directly to Blob using SAS
8. Blob triggers queue message for processing
9. Function processes media, writes results to Cosmos doc
10. Asset becomes visible per privacy rules

UML: GET feed (privacy enforced)

1. Browser → APIM: GET /api/v1/assets?cursor&filters
2. LogicApp validates session + user preferences
3. Query Cosmos for candidate feed docs
4. For private/premium assets, LogicApp checks SQL ACL/entitlements
5. LogicApp filters list; returns only allowed items + URLs
6. Browser loads media via CDN or signed URL

Cloud-Native Media Platform — UML Sequence Diagram

Upload flow and asset retrieval flow using API Management, Logic Apps, Blob Storage, Azure SQL, Cosmos DB, Queue, and Azure Functions





Advanced features | limitations | scalability | conclusion

Advanced features (planned)

- Image similarity search: embeddings + vector search
- Audio recognition (Shazam-like): fingerprinting pipeline + index (future research)
- AI moderation/content safety: classify sensitive/18+
- Recommendations/personalization: feed ranking improvements

Monetization

- Creator monthly subscriptions (premium-only posts)
- Creator payouts after thresholds (followers/engagement)
- Fraud controls: rate limits, anomaly detection, bot prevention

Limitations

- Audio recognition is complex: requires dataset + fingerprinting accuracy concerns
- AI moderation can be wrong: false positives/negatives
- Cost risks: CDN egress, storage growth, AI calls
- Vendor lock-in risks: mitigated via modular interfaces and export paths

Scalability assessment

- Serverless auto-scaling handles traffic spikes
- Blob storage scales for massive media volumes
- Queue decouples processing to prevent overload
- Cosmos DB supports high read throughput for feed/search
- CDN reduces latency and origin load

Conclusion

HyMedia is designed as a cloud-native serverless multimedia platform that supports rich media, strong privacy controls, safety preferences , interactive social features, and a clear roadmap for AI-powered discovery and monetization.

References

- Amazon Web Services (2023) *What is cloud computing?* Available at: <https://aws.amazon.com/what-is-cloud-computing/> (Accessed: 5 March 2026).
- Fielding, R.T. (2000) *Architectural styles and the design of network-based software architectures*. Doctoral dissertation. University of California, Irvine.
- Microsoft (2024) *Azure Blob Storage documentation*. Available at: <https://learn.microsoft.com/en-us/azure/storage/blobs/> (Accessed: 5 March 2026).
- Microsoft (2024) *Azure Cosmos DB documentation*. Available at: <https://learn.microsoft.com/en-us/azure/cosmos-db/> (Accessed: 5 March 2026).
- Microsoft (2024) *Azure Functions documentation*. Available at: <https://learn.microsoft.com/en-us/azure/azure-functions/> (Accessed: 5 March 2026).
- Microsoft (2024) *Azure Logic Apps documentation*. Available at: <https://learn.microsoft.com/en-us/azure/logic-apps/> (Accessed: 5 March 2026).
- Microsoft (2024) *Azure Content Delivery Network (CDN) documentation*. Available at: <https://learn.microsoft.com/en-us/azure/cdn/> (Accessed: 5 March 2026).
- Microsoft (2024) *Designing RESTful web APIs*. Available at: <https://learn.microsoft.com/en-us/azure/architecture/best-practices/api-design> (Accessed: 5 March 2026).
- Newman, S. (2021) *Building microservices: Designing fine-grained systems*. 2nd edn. Sebastopol: O'Reilly Media.
- Pahl, C. (2015) 'Containerization and the PaaS cloud', *IEEE Cloud Computing*, 2(3), pp. 24–31.
- Tariq, Z. and Kamal, R. (2025) *COM682 Cloud Native Development: Module introduction*. Ulster University.
- Ulster University (2025) *COM682 coursework assessment overview*. Ulster University.
- Ulster University (2025) *Cloud computing concepts lecture slides*. Ulster University.
- Zhang, Q., Chen, M. and Li, L. (2010) 'Cloud computing: state-of-the-art and research challenges', *Journal of Internet Services and Applications*, 1(1), pp. 7–18.
- OpenAI, 2026. ChatGPT. [Online] Available at: <https://chat.openai.com/> [Accessed 5 March 2026].

Appendix: Coursework Submission Checklist

General

- Did you include a title slide with project name, one-line description, your name, and student number?
- Is your submission in PowerPoint format and within 15 slides (excluding references)?

Problem & Context

- Did you discuss the problem you are solving?
- Did you identify issues related to scalability of resources in your project?

Solution Design

- Did you include a solution architecture diagram using only Azure resources?
- Did you design and present a multimedia hosting webpage layout (wireframe or design diagram)?
- Did you include a network/architecture diagram showing compute, storage, networking, etc.?
- Did you design a database schema (list of resources and/or ERD diagram) for storing user entries?
- Did you design the REST API for CRUD operations and present it in a UML diagram?
- Did you integrate (or at least propose integration of):
 - Static HTML hosting of a webpage frontend?
 - REST endpoints for service logic and storage connections.
 - Use of both SQL and NoSQL database structures?

Critical Evaluation

- Did you provide an overview of advanced features you intend to develop in the final solution?
- Did you include an assessment of limitations of your designed solution?
- Did you include an assessment of scalability (horizontal/vertical scaling, autoscaling, etc.)?
- Did you justify why you selected specific Azure services over alternatives?

Conclusion & References

- Did you include concluding comments summarising your design?
- Did you provide references to sources, frameworks, or Azure documentation you used?

✓ If you can tick all these boxes, your submission is likely to meet the expectations.